

Guide de Gestion du Modele CNN

Entrainement, Reentrainement et Ajout de Donnees

Application APPIN

Auteur : Julien Guibert - INRAe Toxalim / MetaboHUB
Version 1.1 - Mars 2026

Table des matieres

1. Vue d'ensemble
2. Structure du dossier CNN_training
3. Ajout de spectres reels
4. Format des fichiers d'annotation
5. Generation de spectres synthetiques
6. Workflow de reentrainement
7. Parametres d'entrainement
8. Fine-tuning vs Entrainement from scratch
9. Evaluation et deploiement
10. Depannage
11. Reference des fonctions

1. Vue d'ensemble

1.1 Objectif du CNN

Le CNN (Convolutional Neural Network) d'APPIN est un reseau de neurones 1D entraine pour detecter les pics dans des spectres RMN 2D. Il fonctionne en :

1. Decomposant le spectre 2D en coupes 1D (lignes et colonnes)
2. Analysant chaque coupe 1D avec le CNN
3. Classifiant chaque point comme : fond (0), bord de pic (1), ou centre de pic (2)
4. Recombinant les detections en pics 2D avec clustering DBSCAN

1.2 Architecture du modele

Composant	Specification
Input	2048 points (spectre 1D)
Couches	8 conv1D (filtres: 40->20->10->20->10->30->18->18)
Activation	ReLU
Output 1	Classification 3 classes (softmax)
Output 2	Regression 3 valeurs (ppm, intensite, FWHH)
Loss	Focal Loss (classification) + MSE (regression)

1.3 Donnees d'entrainement

Le CNN est entraine sur un mix de spectres synthetiques (generes a partir de bases de donnees de metabolites) et de spectres reels (annotes manuellement avec les positions des pics).

2. Structure du dossier CNN_training

```
CNN_training/
  spectres_reels/          # Spectres Bruker a annoter
    TOCSY/
      spectre_001/pdata/1/ # Structure Bruker standard
      spectre_002/pdata/1/
    HSQC/
    COSY/
    UFCOSY/
  annotations/             # Fichiers CSV des pics
    TOCSY_spectre_001_peaks.csv
    HSQC_spectre_001_peaks.csv
  saved_model/             # Poids du modele
    weights/               # Poids actuels (production)
    weights_v2/            # Nouvelle version (test)
    weights_hsqc/          # Modele HSQC specifique
    checkpoints/           # Checkpoints d'entrainement
    training_history.rds   # Historique d'entrainement
  CNN_learn.R              # Script principal d'entrainement
  CNN_train_HSQC.R         # Script entrainement HSQC
  spectrum_generator_random.R # Generateur de spectres synthetiques
```

3. Ajout de spectres reels

3.1 Prerequis

Pour ajouter un nouveau spectre d'entrainement, vous avez besoin de :

- Donnees Bruker : dossier contenant pdata/1/ avec les fichiers 2rr, procs, proc2s
- Annotation CSV : fichier listant les positions des pics (voir section 4)

3.2 Etapes d'ajout

Etape 1 : Copier le spectre Bruker

Placez le dossier du spectre dans le sous-dossier approprié :

```
spectres_reels/TOCSY/mon_nouveau_spectre/pdata/1/
```

Etape 2 : Creer l'annotation

Deux methodes possibles :

Methode A : Via l'application APPIN

1. Charger le spectre dans l'application
2. Utiliser la detection automatique (Local Max ou CNN existant)
3. Corriger manuellement les detections
4. Exporter les pics en CSV via '6. Save & Export'

Methode B : Annotation manuelle

1. Ouvrir le spectre dans un logiciel externe (TopSpin, MestReNova, etc.)
2. Noter les positions ppm de chaque pic
3. Creer un fichier CSV au format requis

Etape 3 : Nommer le fichier d'annotation

Convention de nommage : {TYPE}_{NOM_SPECTRE}_peaks.csv

Exemples : TOCSY_MTH_Melange_001_peaks.csv, HSQC_Urine_Sample_042_peaks.csv

4. Format des fichiers d'annotation

4.1 Colonnes requises

Colonne	Type	Description	Obligatoire
F1_ppm	numeric	Position ppm sur l'axe F1	Oui
F2_ppm	numeric	Position ppm sur l'axe F2	Oui
intensity	numeric	Intensite relative du pic	Non (default: 1.0)

4.2 Formats acceptes

Format international (recommande) :

```
F1_ppm,F2_ppm,intensity  
3.456,1.234,0.85  
4.123,2.567,0.42
```

Format francais (point-virgule) :

```
F1_ppm;F2_ppm;intensity  
3,456;1,234;0,85  
4,123;2,567;0,42
```

4.3 Noms de colonnes alternatifs acceptes

Colonne cible	Variantes acceptees
F1_ppm	F1.ppm, f1_ppm, F1, ppm1, x_ppm
F2_ppm	F2.ppm, f2_ppm, F2, ppm2, y_ppm
intensity	Intensity, int, amplitude, height

5. Generation de spectres synthétiques

5.1 Deux types de spectres synthétiques

Le générateur produit deux types de spectres pour un entraînement robuste :

Type A : Spectres basés sur une BdD de métabolites

Utilise `BdDReference_CPMG` (ou `BdDReference_13C`) pour simuler des mélanges de métabolites réels.

```
res <- generate_random_spectrum_with_peaks(  
  BdD = BdDReference_CPMG,  
  min_compounds = 20,  
  max_compounds = 34,  
  noise_sd = 0.01,  
  seed = 42  
)
```

Type B : Coupes TOCSY denses

Simule des coupes 1D avec de nombreux multiplets denses, typiques des spectres TOCSY.

```
res <- generate_dense_tocsy_1D_cut(  
  n_points = 2048,  
  n_multiplets = 100,  
  max_multiplet_order = 6,  
  noise_level = 0.000001,  
  ppm_range = c(0, 14),  
  coupling_range = c(4, 14),  
  seed = 42,  
  plot = FALSE  
)
```

5.2 Generation d'un dataset mixte

La fonction `generate_spectrum_labels_full_mixed()` combine les deux types :

```
data_synthetic <- generate_spectrum_labels_full_mixed(  
  n_spectra = 5000,  
  n_points = 2048,  
  intensity_threshold = 0.2,  
  center_margin = 0.2,  
  mix_ratio = 0.5, # 50% TOCSY, 50% BdD  
  seed = 123  
)  
  
# Structure de sortie  
# data_synthetic$X      [5000, 2048]    Spectres  
# data_synthetic$y_class [5000, 2048]    Labels de classification  
# data_synthetic$y_reg   [5000, 2048, 3] Labels de regression
```

6. Workflow de reentrainement

6.1 Vue d'ensemble

1. PREPARATION : Organiser les spectres Bruker, Créer les annotations CSV
2. TRAITEMENT DES SPECTRES REELS : Charger, Decomposer en coupes 1D, Generer les labels
3. GENERATION SYNTHETIQUE : Créer 5000+ spectres synthétiques
4. FUSION DES DATASETS : Combiner reels + synthétiques
5. ENTRAINEMENT : Configurer callbacks, Lancer (~2-4h)
6. EVALUATION : Visualiser les metriques, Tester sur spectres non vus
7. DEPLOIEMENT : Remplacer les poids dans saved_model/weights

6.2 Script d'entrainement avec fine-tuning

NOUVEAU dans v1.1 : L'option `use_pretrained` permet de choisir entre fine-tuning (continuer l'apprentissage) et entrainement from scratch (repartir de zero).

```
# === OPTION FINE-TUNING ===
# Mettre a TRUE pour continuer l'entrainement a partir des poids existants
# Mettre a FALSE pour entrainer depuis zero (poids aleatoires)
use_pretrained <- TRUE

model <- build_peak_predictor()

# Fine-tuning : charger les poids pre-entraînés si demande
if (use_pretrained) {
  pretrained_path <- "saved_model/weights"
  if (dir.exists(pretrained_path) ||
      file.exists(paste0(pretrained_path, ".index"))) {
    cat("Chargement des poids pre-entraînés pour fine-tuning...\n")
    tryCatch({
      load_model_weights_tf(model, pretrained_path)
      cat("Poids pre-entraînés chargés\n")
    }, error = function(e) {
      cat("Impossible de charger les poids, entrainement depuis zero\n")
    })
  }
} else {
  cat("Entrainement depuis zero (use_pretrained = FALSE)\n")
}

# Entrainement
model %>% fit(
  x = X_train,
  y = list(class_output = y_class_train, reg_output = y_reg_train),
  epochs = 200,
  batch_size = 32,
  validation_split = 0.2,
  callbacks = callbacks_list
)

# Sauvegarder (dans un dossier versionné pour ne pas écraser)
save_model_weights_tf(model, "saved_model/weights_new")
```


7. Parametres d'entrainement

7.1 Parametres recommandes

Parametre	Valeur	Description
epochs	200	Nombre max d'epoques (early stopping actif)
batch_size	32	Taille des mini-batches
validation_split	0.2	20% des donnees pour validation
learning_rate	1e-4	Taux d'apprentissage initial
early_stopping_patience	50	Epoques sans amelioration avant arret
lr_reduce_patience	20	Epoques avant reduction du LR
lr_reduce_factor	0.5	Facteur de reduction du LR

7.2 Focal Loss

La focal loss est utilisee pour gerer le desequilibre de classes (beaucoup plus de points 'fond' que de 'pics') :

Parametre	Valeur	Effet
gamma	3.0	Focus sur les exemples difficiles
alpha	1.5	Poids des classes minoritaires

7.3 Labels de classification

Classe	Signification	Description
0	Background	Point sans pic
1	Pic fort centre	Pic avec intensite $\geq$ seuil ET dans la zone centrale
2	Pic faible/bord	Pic avec intensite $<$ seuil OU hors zone centrale

8. Fine-tuning vs Entrainement from scratch

8.1 Differences

Aspect	Fine-tuning (use_pretrained=TRUE)	From scratch (use_pretrained=FALSE)
Poids initiaux	Charges depuis saved_model/weights	Aléatoires
Apprentissage	Continue/ameliore l'existant	Repart de zero
Temps	Plus rapide (convergence)	Plus long
Risque	Catastrophic forgetting possible	Perte de tout l'ancien
Cas d'usage	Ajout de nouveaux spectres	Changement majeur d'architecture

8.2 Quand utiliser quoi ?

Utiliser le fine-tuning (use_pretrained = TRUE) quand :

- Vous ajoutez quelques nouveaux spectres annotés
- Vous voulez améliorer la détection sur un type de spectre spécifique
- Le modèle actuel fonctionne mais peut être amélioré

Utiliser l'entraînement from scratch (use_pretrained = FALSE) quand :

- Vous changez l'architecture du modèle
- Le dataset d'entraînement change radicalement
- Le modèle actuel a de graves problèmes

8.3 Bonnes pratiques de versionnement

Toujours sauvegarder les nouveaux poids dans un dossier versionné avant de remplacer :

```
# Sauvegarder dans un nouveau dossier
save_model_weights_tf(model, "saved_model/weights_v2")

# Après validation, deployer en production
file.rename("saved_model/weights", "saved_model/weights_backup")
file.rename("saved_model/weights_v2", "saved_model/weights")
```

9. Evaluation et deployment

9.1 Metriques a surveiller

Metrique	Bon signe	Mauvais signe
val_loss	Diminue puis se stabilise	Augmente (overfitting)
class_output_accuracy	> 0.95	< 0.90
Training time	2-4h	> 8h (probleme de donnees)

9.2 Test sur spectre reel

```
# Charger le nouveau modele
test_model <- build_peak_predictor()
load_model_weights_tf(test_model, "saved_model/weights_v2")

# Tester sur un spectre
test_spectrum <- read Bruker(dir = "chemin/vers/spectre/pdata/1", dim = "2D")
params <- list(
  pred_class_thres = 0.3,
  int_thres = 0,
  eps_value = 0.0068,
  trace_filter_ratio = 0.5
)
results <- run_cnn_peak_picking(
  rr_norm = normalize_spectrum(test_spectrum$spectrumData),
  model = test_model,
  params = params,
  method = "batch"
)
cat("Pics detectes:", nrow(results$peaks), "\n")
```

9.3 Deployment en production

```
# 1. Sauvegarder l'ancien modele
file.rename("saved_model/weights", "saved_model/weights_backup")

# 2. Deployer le nouveau modele
file.rename("saved_model/weights_v2", "saved_model/weights")

# 3. Tester dans l'application APPIN
```

10. Depannage

10.1 Problemes courants

Probleme	Cause probable	Solution
Erreur de lecture CSV	Format incorrect	Verifier separateur (, ou ;)
Colonnes manquantes	Noms non reconnus	Utiliser F1_ppm, F2_ppm
Pas de pics detectes	Seuil trop eleve	Baisser pred_class_thres a 0.2
Trop de faux positifs	Entrainement insuffisant	Ajouter plus de spectres
Out of memory	Dataset trop grand	Reduire n_spectra ou batch_size
Entrainement tres long	GPU non utilise	Verifier TensorFlow-GPU

10.2 Verification des annotations

```
check_annotation <- function(csv_path) {  
  df <- read_csv_auto(csv_path)  
  cat("Fichier:", csv_path, "\n")  
  cat("Colonnes:", paste(names(df), collapse = ", "), "\n")  
  cat("Nombre de pics:", nrow(df), "\n")  
  if ("F1_ppm" %in% names(df)) {  
    cat("F1_ppm range:", range(df$F1_ppm), "\n")  
  }  
  return(df)  
}
```

11. Reference des fonctions

11.1 Generation de spectres

Fonction	Description
generate_random_spectrum_with_peakset()	Genere un spectre a partir d'une BdD
generate_dense_tocsy_1D_cut()	Genere une coupe TOCSY dense synthetique
generate_spectrum_labels_full()	Genere un dataset de spectres BdD avec labels
generate_spectrum_labels_full_mixed()	Genere un dataset mixte (BdD + TOCSY)

11.2 Traitement de spectres reels

Fonction	Description
read_csv_auto()	Lecture robuste de CSV (detection auto du format)
load_2D_peaks_robust()	Chargement d'annotations avec gestion des variantes
decompose_2D_to_1D_training()	Decomposition d'un spectre 2D en coupes 1D
scan_spectres_folder()	Scan automatique d'un dossier de spectres
process_all_real_spectra()	Traitement de tous les spectres reels
merge_real_and_synthetic()	Fusion des datasets reels et synthetiques

11.3 Modele CNN

Fonction	Description
build_peak_predictor()	Construit l'architecture du CNN
load_model_weights_tf()	Charge les poids d'un modele
save_model_weights_tf()	Sauvegarde les poids d'un modele

Checklist de reentrainement

1. Organiser les spectres Bruker dans spectres_reels/{TYPE}/
2. Creer les fichiers CSV d'annotation (F1_ppm, F2_ppm, intensity)
3. Verifier les annotations avec check_annotation()
4. Configurer use_pretrained selon le besoin (TRUE pour fine-tuning)
5. Executer process_all_real_spectra()

6. Generer les spectres synthetiques
7. Fusionner avec merge_real_and_synthetic()
8. Lancer l'entrainement
9. Evaluer les metriques
10. Tester sur des spectres non vus
11. Si OK, deployer dans saved_model/weights
12. Tester dans l'application APPIN

Guide de Gestion du Modele CNN - v1.1
APPIN - INRAe Toxalim / MetaboHUB